

ASSIGNMENT 8.2

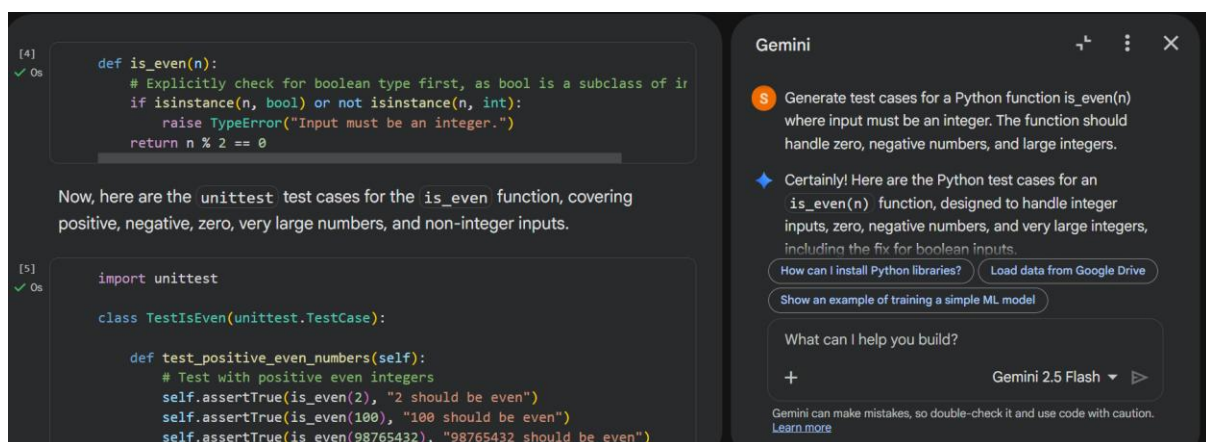
SHRUTHI ADDAGUDI | 2303A52133 | BATCH 37

Task 1 – Test-Driven Development for Even/Odd Number Validator

Use AI tools to first generate test cases for a function `is_even(n)` and then implement the function so that it satisfies all generated tests

Prompt used:

Generate test cases for a Python function `is_even(n)` where input must be an integer. The function should handle zero, negative numbers, and large integers.

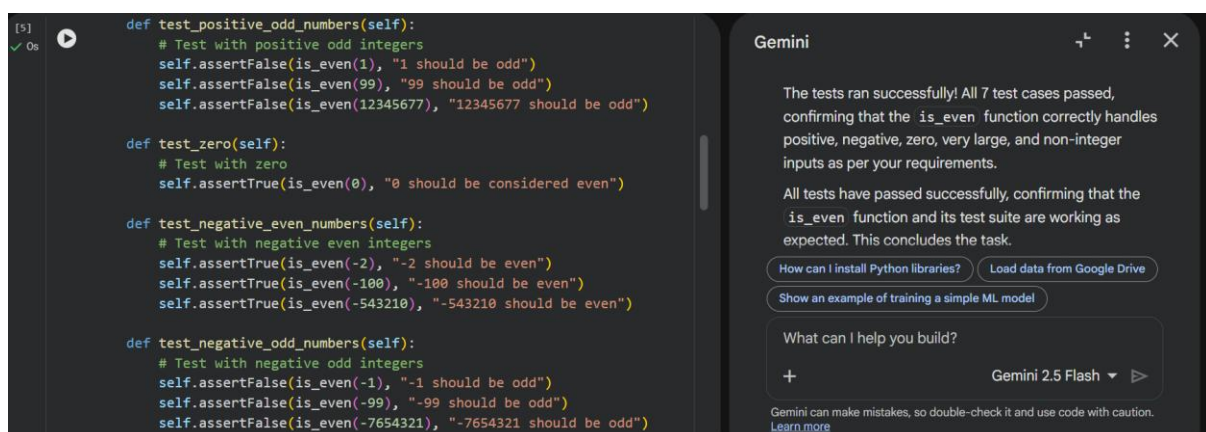


The screenshot shows a code editor on the left and the Gemini chat interface on the right. The code editor contains the following Python code:

```
[4]
✓ Os
def is_even(n):
    # Explicitly check for boolean type first, as bool is a subclass of int
    if isinstance(n, bool) or not isinstance(n, int):
        raise TypeError("Input must be an integer.")
    return n % 2 == 0
```

Below the code, a text prompt reads: "Now, here are the unittest test cases for the `is_even` function, covering positive, negative, zero, very large numbers, and non-integer inputs."

The Gemini chat interface on the right shows the prompt: "Generate test cases for a Python function `is_even(n)` where input must be an integer. The function should handle zero, negative numbers, and large integers." The response from Gemini is: "Certainly! Here are the Python test cases for an `is_even(n)` function, designed to handle integer inputs, zero, negative numbers, and very large integers, including the fix for boolean inputs."



The screenshot shows a code editor on the left and the Gemini chat interface on the right. The code editor contains the following Python code:

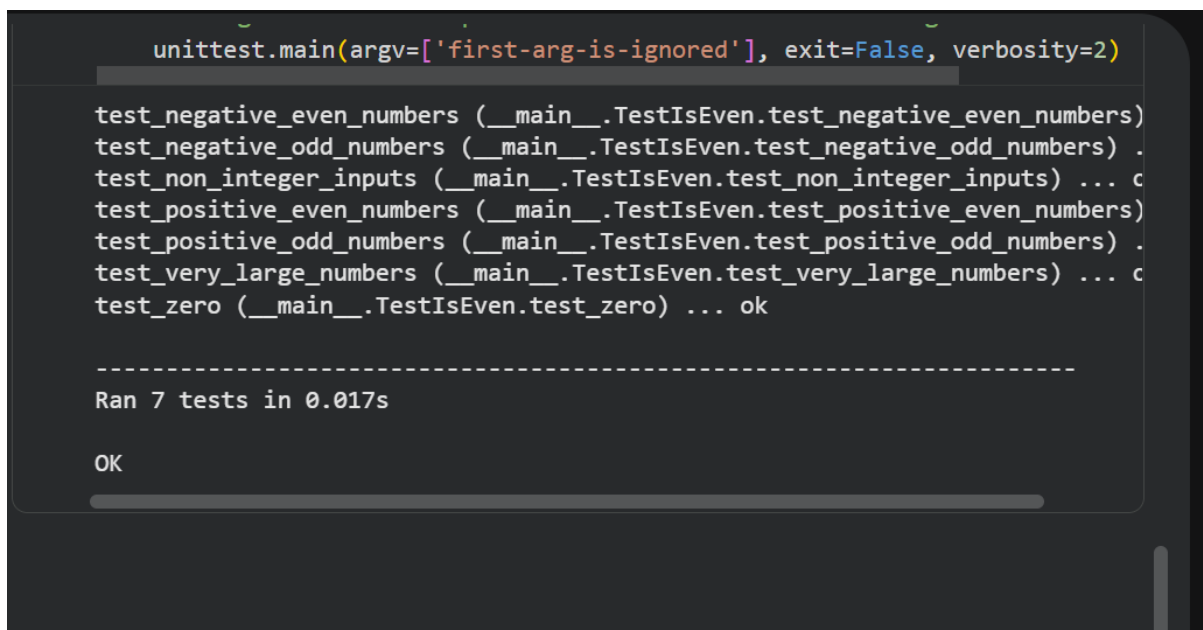
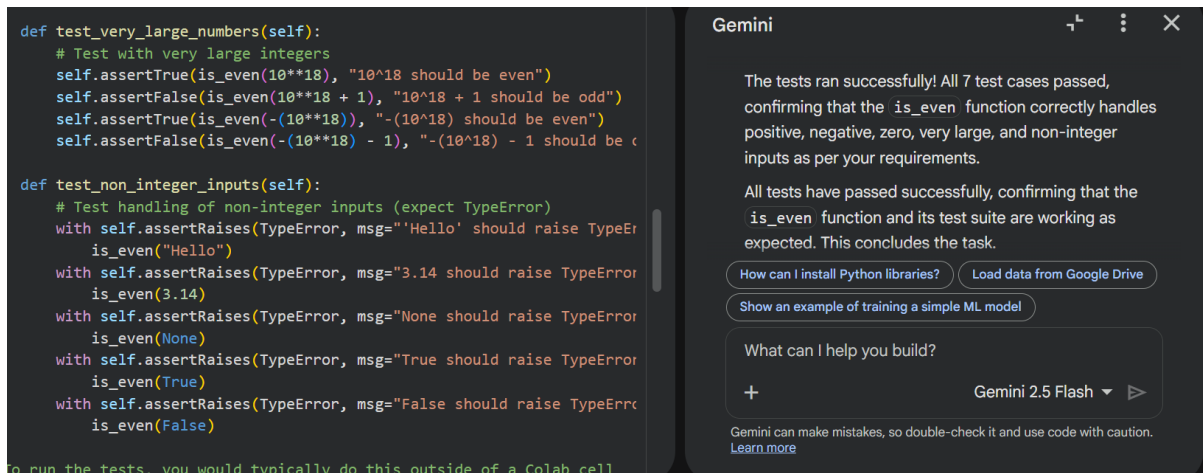
```
[5]
✓ Os
def test_positive_odd_numbers(self):
    # Test with positive odd integers
    self.assertFalse(is_even(1), "1 should be odd")
    self.assertFalse(is_even(99), "99 should be odd")
    self.assertFalse(is_even(12345677), "12345677 should be odd")

def test_zero(self):
    # Test with zero
    self.assertTrue(is_even(0), "0 should be considered even")

def test_negative_even_numbers(self):
    # Test with negative even integers
    self.assertTrue(is_even(-2), "-2 should be even")
    self.assertTrue(is_even(-100), "-100 should be even")
    self.assertTrue(is_even(-543210), "-543210 should be even")

def test_negative_odd_numbers(self):
    # Test with negative odd integers
    self.assertFalse(is_even(-1), "-1 should be odd")
    self.assertFalse(is_even(-99), "-99 should be odd")
    self.assertFalse(is_even(-7654321), "-7654321 should be odd")
```

The Gemini chat interface on the right shows the prompt: "The tests ran successfully! All 7 test cases passed, confirming that the `is_even` function correctly handles positive, negative, zero, very large, and non-integer inputs as per your requirements." The response from Gemini is: "All tests have passed successfully, confirming that the `is_even` function and its test suite are working as expected. This concludes the task."



Code Explanation

- `isinstance()` ensures input validation.
- `% 2 == 0` checks divisibility by 2.
- Works for positive, negative, and zero values.

Task Description

Task 2 – Test-Driven Development for String Case Converter

- Ask AI to generate test cases for two functions:

- to_uppercase(text)
- to_lowercase(text)

Prompt used:

Generate test cases for uppercase and lowercase conversion functions that handle empty strings, mixed case, and invalid inputs such as numbers or None.

```

[6]
✓ Os
def to_uppercase(s):
    if not isinstance(s, str):
        raise TypeError("Input must be a string.")
    return s.upper()

def to_lowercase(s):
    if not isinstance(s, str):
        raise TypeError("Input must be a string.")
    return s.lower()

```

Now, here are the `unittest` test cases for both functions, covering various scenarios.

```

[7]
✓ Os
import unittest

class TestStringConversion(unittest.TestCase):

```

Generate test cases for uppercase and lowercase conversion functions that handle empty strings, mixed case, and invalid inputs such as numbers or None.

Certainly! Here are the Python test cases for `to_uppercase` and `to_lowercase` functions, designed to handle empty strings, mixed case, and invalid inputs.

Call the `to_uppercase` function with an example string

Add docstrings to the `to_uppercase` and `to_lowercase` functions

Generate a new Python function to count vowels

What can I help you build?

+ Gemini 2.5 Flash ▶

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

```

import unittest

class TestStringConversion(unittest.TestCase):

    def test_empty_string(self):
        # Test with empty strings
        self.assertEqual(to_uppercase(""), "", "Empty string to uppercase should be empty")
        self.assertEqual(to_lowercase(""), "", "Empty string to lowercase should be empty")

    def test_mixed_case_to_uppercase(self):
        # Test converting mixed case to uppercase
        self.assertEqual(to_uppercase("HelloWorld"), "HELLOWORLD")
        self.assertEqual(to_uppercase("python PROGRAMMING"), "PYTHON PROGRAMMING")
        self.assertEqual(to_uppercase("aBcDeFg"), "ABCDEFGG")

    def test_mixed_case_to_lowercase(self):
        # Test converting mixed case to lowercase
        self.assertEqual(to_lowercase("HelloWorld"), "helloworld")
        self.assertEqual(to_lowercase("Python PROGRAMMING"), "python programming")
        self.assertEqual(to_lowercase("AbCdEfG"), "abcdefg")

    def test_already_uppercase(self):
        # Test with strings that are already uppercase
        self.assertEqual(to_uppercase("HELLO"), "HELLO")
        self.assertEqual(to_uppercase("PYTHON"), "PYTHON")

```

```

def test_already_lowercase(self):
    # Test with strings that are already lowercase
    self.assertEqual(to_lowercase("hello"), "hello")
    self.assertEqual(to_lowercase("python"), "python")

def test_strings_with_numbers_and_symbols(self):
    # Test strings containing numbers and symbols (should remain unchanged)
    self.assertEqual(to_uppercase("123!@#ABC"), "123!@#ABC")
    self.assertEqual(to_lowercase("123!@#abc"), "123!@#abc")
    self.assertEqual(to_uppercase("MiXeD123_SyMbOlS"), "MIXED123_SYMBOLS")
    self.assertEqual(to_lowercase("MiXeD123_SyMbOlS"), "mixed123_symbols")

def test_non_string_inputs_uppercase(self):
    # Test handling of non-string inputs for to_uppercase (expect TypeError)
    with self.assertRaises(TypeError, msg="None should raise TypeError"):
        to_uppercase(None)
    with self.assertRaises(TypeError, msg="Number should raise TypeError"):
        to_uppercase(123)
    with self.assertRaises(TypeError, msg="Float should raise TypeError"):
        to_uppercase(3.14)
    with self.assertRaises(TypeError, msg="List should raise TypeError"):
        to_uppercase(["a", "b"])

def test_non_string_inputs_lowercase(self):
    # Test handling of non-string inputs for to_lowercase (expect TypeError)
    with self.assertRaises(TypeError, msg="None should raise TypeError"):

```

```

        to_lowercase(3.14)
    with self.assertRaises(TypeError, msg="List should raise TypeError"):
        to_lowercase(["a", "b"])

# To run the tests in Colab
if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False, verbosity=2)

... test_negative_even_numbers (__main__.TestIsEven.test_negative_even_numbers) ... ok
test_negative_odd_numbers (__main__.TestIsEven.test_negative_odd_numbers) ... ok
test_non_integer_inputs (__main__.TestIsEven.test_non_integer_inputs) ... ok
test_positive_even_numbers (__main__.TestIsEven.test_positive_even_numbers) ... ok
test_positive_odd_numbers (__main__.TestIsEven.test_positive_odd_numbers) ... ok
test_very_large_numbers (__main__.TestIsEven.test_very_large_numbers) ... ok
test_zero (__main__.TestIsEven.test_zero) ... ok
test_already_lowercase (__main__.TestStringConversion.test_already_lowercase) ... ok
test_already_uppercase (__main__.TestStringConversion.test_already_uppercase) ... ok
test_empty_string (__main__.TestStringConversion.test_empty_string) ... ok
test_mixed_case_to_lowercase (__main__.TestStringConversion.test_mixed_case_to_lowercase) ... ok
test_mixed_case_to_uppercase (__main__.TestStringConversion.test_mixed_case_to_uppercase) ... ok
test_non_string_inputs_lowercase (__main__.TestStringConversion.test_non_string_inputs_lowercase)
test_non_string_inputs_uppercase (__main__.TestStringConversion.test_non_string_inputs_uppercase)
test_strings_with_numbers_and_symbols (__main__.TestStringConversion.test_strings_with_numbers_and_symbols) ... ok

-----
Ran 15 tests in 0.021s

```

Code Explanation

- Validates input type.

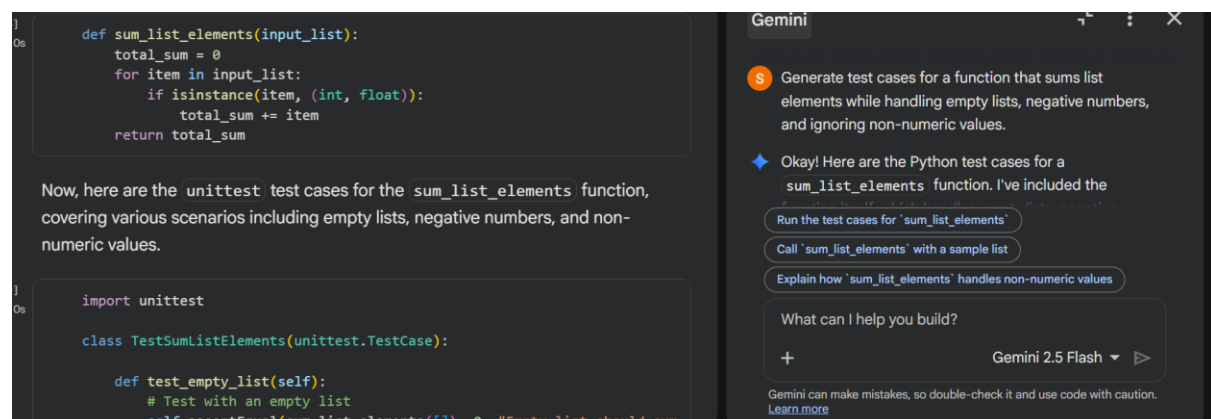
- Uses built-in string methods for conversion.
- Handles empty strings safely.

Task 3 — Test Driven Development for List Sum Calculator

Use AI to generate test cases for a function `sum_list` (numbers) that calculates the sum of list elements.

Prompt used:

Generate test cases for a function that sums list elements while handling empty lists, negative numbers, and ignoring non-numeric values.



```

def test_negative_numbers(self):
    # Test with a list of negative numbers
    self.assertEqual(sum_list_elements([-1, -2, -3, -4, -5]), -15, "Sum of negative numbers")

def test_mixed_numbers(self):
    # Test with a list of mixed positive and negative numbers
    self.assertEqual(sum_list_elements([-1, 2, -3, 4, -5]), -3, "Sum of mixed positive and neg

def test_zero(self):
    # Test with a list containing zero
    self.assertEqual(sum_list_elements([0, 0, 0]), 0, "Sum of zeros")
    self.assertEqual(sum_list_elements([1, 0, -1]), 0, "Sum with zero included")

def test_floats(self):
    # Test with floating-point numbers
    self.assertAlmostEqual(sum_list_elements([1.5, 2.5, 3.0]), 7.0, "Sum of floats")
    self.assertAlmostEqual(sum_list_elements([-1.5, 2.5]), 1.0, "Sum of mixed floats")

def test_mixed_numeric_and_non_numeric(self):
    # Test with non-numeric values that should be ignored
    self.assertEqual(sum_list_elements([1, 'a', 2, None, 3, True, 4.5]), 10.5, "Non-numeric va
    self.assertEqual(sum_list_elements(['x', 'y', 10, False, 20]), 30, "Non-numeric and boolea

def test_only_non_numeric(self):
    # Test with a list containing only non-numeric values
    self.assertEqual(sum list elements(['a', 'b', None, {}]), 0, "List with only non-numeric s

```

```

... test_negative_even_numbers (__main__.TestIsEven.test_negative_even_numbers) ... ok
test_negative_odd_numbers (__main__.TestIsEven.test_negative_odd_numbers) ... ok
test_non_integer_inputs (__main__.TestIsEven.test_non_integer_inputs) ... ok
test_positive_even_numbers (__main__.TestIsEven.test_positive_even_numbers) ... ok
test_positive_odd_numbers (__main__.TestIsEven.test_positive_odd_numbers) ... ok
test_very_large_numbers (__main__.TestIsEven.test_very_large_numbers) ... ok
test_zero (__main__.TestIsEven.test_zero) ... ok
test_already_lowercase (__main__.TestStringConversion.test_already_lowercase) ... ok
test_already_uppercase (__main__.TestStringConversion.test_already_uppercase) ... ok
test_empty_string (__main__.TestStringConversion.test_empty_string) ... ok
test_mixed_case_to_lowercase (__main__.TestStringConversion.test_mixed_case_to_lowercase) ... ok
test_mixed_case_to_uppercase (__main__.TestStringConversion.test_mixed_case_to_uppercase) ... ok
test_non_string_inputs_lowercase (__main__.TestStringConversion.test_non_string_inputs_lowercase)
test_non_string_inputs_uppercase (__main__.TestStringConversion.test_non_string_inputs_uppercase)
test_strings_with_numbers_and_symbols (__main__.TestStringConversion.test_strings_with_numbers_and_symbols)
test_empty_list (__main__.TestSumListElements.test_empty_list) ... ok
test_floats (__main__.TestSumListElements.test_floats) ... ok
test_large_numbers (__main__.TestSumListElements.test_large_numbers) ... ok
test_mixed_numbers (__main__.TestSumListElements.test_mixed_numbers) ... ok
test_mixed_numeric_and_non_numeric (__main__.TestSumListElements.test_mixed_numeric_and_non_numeric)
test_negative_numbers (__main__.TestSumListElements.test_negative_numbers) ... ok
test_only_non_numeric (__main__.TestSumListElements.test_only_non_numeric) ... ok
test_positive_numbers (__main__.TestSumListElements.test_positive_numbers) ... ok
test_zero (__main__.TestSumListElements.test_zero) ... ok

=====
FAIL: test mixed numeric and non numeric ( __main__.TestSumListElements.test mixed numeric and non

```

Code Explanation

- Validates list input.
- Ignores non-numeric values safely.

- Works for negative numbers and empty lists.

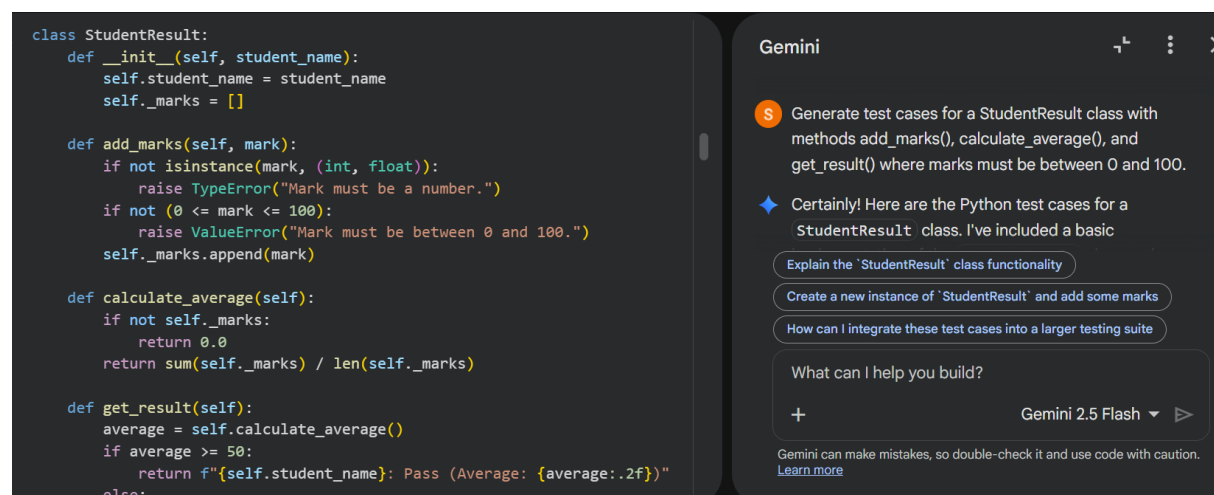
Task 4 – Test Cases for Student Result Class

- Generate test cases for a StudentResult class with the following

methods:

- add_marks(mark)
- calculate_average()
- get_result()

Prompt used:



The image shows a side-by-side comparison of a Python class definition and a Gemini chat interface. On the left, a code editor displays the following Python code for a `StudentResult` class:

```
class StudentResult:
    def __init__(self, student_name):
        self.student_name = student_name
        self._marks = []

    def add_marks(self, mark):
        if not isinstance(mark, (int, float)):
            raise TypeError("Mark must be a number.")
        if not (0 <= mark <= 100):
            raise ValueError("Mark must be between 0 and 100.")
        self._marks.append(mark)

    def calculate_average(self):
        if not self._marks:
            return 0.0
        return sum(self._marks) / len(self._marks)

    def get_result(self):
        average = self.calculate_average()
        if average >= 50:
            return f"{self.student_name}: Pass (Average: {average:.2f})"
        else:
```

On the right, the Gemini chat interface is shown with the following content:

Gemini

Generate test cases for a StudentResult class with methods add_marks(), calculate_average(), and get_result() where marks must be between 0 and 100.

Certainly! Here are the Python test cases for a StudentResult class. I've included a basic

Explain the 'StudentResult' class functionality

Create a new instance of 'StudentResult' and add some marks

How can I integrate these test cases into a larger testing suite

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)


```

class TestStudentResult(unittest.TestCase):

    def setUp(self):
        # Set up a new StudentResult instance before each test
        self.student = StudentResult("Alice")

    def test_add_valid_marks(self):
        # Test adding valid marks
        self.student.add_marks(75)
        self.student.add_marks(90)
        self.assertEqual(self.student._marks, [75, 90])
        self.student.add_marks(0)
        self.student.add_marks(100)
        self.assertEqual(self.student._marks, [75, 90, 0, 100])

    def test_add_invalid_marks_type(self):
        # Test adding non-numeric marks
        with self.assertRaises(TypeError, msg="String mark should raise TypeError"):
            self.student.add_marks("85")
        with self.assertRaises(TypeError, msg="None mark should raise TypeError"):
            self.student.add_marks(None)
        with self.assertRaises(TypeError, msg="List mark should raise TypeError"):
            self.student.add_marks([80])

    def test_add_invalid_marks_range(self):
        # Test adding marks out of range (0-100)

```

```

    def test_add_invalid_marks_range(self):
        # Test adding marks out of range (0-100)
        with self.assertRaises(ValueError, msg="Mark > 100 should raise ValueError"):
            self.student.add_marks(101)
        with self.assertRaises(ValueError, msg="Mark < 0 should raise ValueError"):
            self.student.add_marks(-1)

    def test_calculate_average_with_marks(self):
        # Test average calculation with valid marks
        self.student.add_marks(70)
        self.student.add_marks(80)
        self.student.add_marks(90)
        self.assertAlmostEqual(self.student.calculate_average(), 80.0)

    def test_calculate_average_no_marks(self):
        # Test average calculation with no marks added
        self.assertEqual(self.student.calculate_average(), 0.0)

    def test_calculate_average_with_floats(self):
        # Test average calculation with float marks
        self.student.add_marks(75.5)
        self.student.add_marks(85.0)
        self.assertAlmostEqual(self.student.calculate_average(), 80.25)

    def test_get_result_pass(self):
        # Test get result for a passing student

```



```

        self.student.add_marks(60)
        self.student.add_marks(70)
        self.assertEqual(self.student.get_result(), "Alice: Pass (Average: 65.00)")

    def test_get_result_fail(self):
        # Test get_result for a failing student
        self.student.add_marks(30)
        self.student.add_marks(40)
        self.assertEqual(self.student.get_result(), "Alice: Fail (Average: 35.00)")

    def test_get_result_borderline_pass(self):
        # Test get_result for a student with average exactly 50
        self.student.add_marks(50)
        self.assertEqual(self.student.get_result(), "Alice: Pass (Average: 50.00)")

    def test_get_result_no_marks(self):
        # Test get_result when no marks are added (average 0)
        self.assertEqual(self.student.get_result(), "Alice: Fail (Average: 0.00)")

# To run the tests in Colab
if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False, verbosity=2)

```

```

test_negative_even_numbers (__main__.TestIsEven.test_negative_even_numbers) ... ok
test_negative_odd_numbers (__main__.TestIsEven.test_negative_odd_numbers) ... ok
test_non_integer_inputs (__main__.TestIsEven.test_non_integer_inputs) ... ok

```

```

test_calculate_average_with_floats (__main__.TestStudentResult.test_calculate_average_with_floats) ... ok
test_calculate_average_with_marks (__main__.TestStudentResult.test_calculate_average_with_marks) ... ok
test_get_result_borderline_pass (__main__.TestStudentResult.test_get_result_borderline_pass) ... ok
test_get_result_fail (__main__.TestStudentResult.test_get_result_fail) ... ok
test_get_result_no_marks (__main__.TestStudentResult.test_get_result_no_marks) ... ok
test_get_result_pass (__main__.TestStudentResult.test_get_result_pass) ... ok
test_empty_list (__main__.TestSumListElements.test_empty_list) ... ok
test_floats (__main__.TestSumListElements.test_floats) ... ok
test_large_numbers (__main__.TestSumListElements.test_large_numbers) ... ok
test_mixed_numbers (__main__.TestSumListElements.test_mixed_numbers) ... ok
test_mixed_numeric_and_non_numeric (__main__.TestSumListElements.test_mixed_numeric_and_non_numeric) ... FAIL
test_negative_numbers (__main__.TestSumListElements.test_negative_numbers) ... ok
test_only_non_numeric (__main__.TestSumListElements.test_only_non_numeric) ... ok
test_positive_numbers (__main__.TestSumListElements.test_positive_numbers) ... ok
test_zero (__main__.TestSumListElements.test_zero) ... ok

```

```

=====
FAIL: test_mixed_numeric_and_non_numeric (__main__.TestSumListElements.test_mixed_numeric_and_non_numeric)
-----
Traceback (most recent call last):
  File "/tmp/ipython-input-3362737179.py", line 33, in test_mixed_numeric_and_non_numeric
    self.assertEqual(sum_list_elements([1, 'a', 2, None, 3, True, 4.5]), 10.5, "Non-numeric values should be ignored")
AssertionError: 11.5 != 10.5 : Non-numeric values should be ignored

-----
Ran 34 tests in 0.051s

FAILED (failures=1)

```

Code Explanation

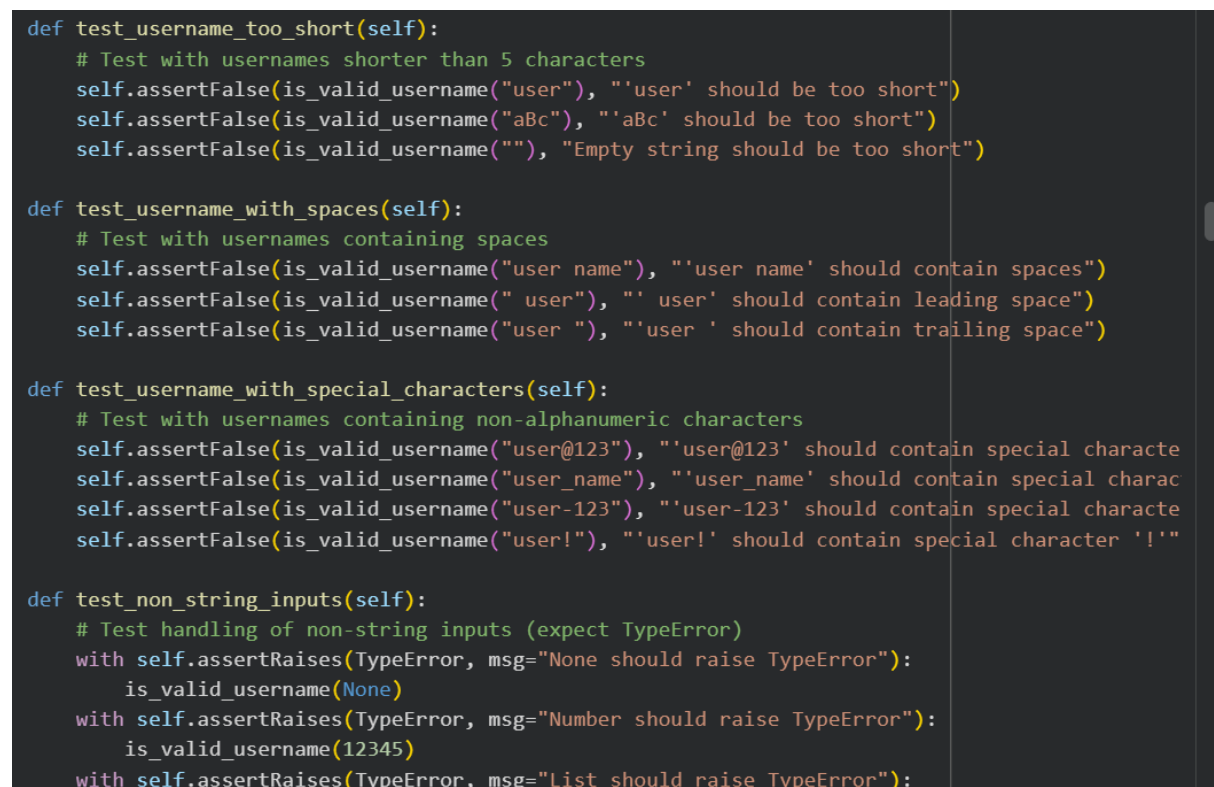
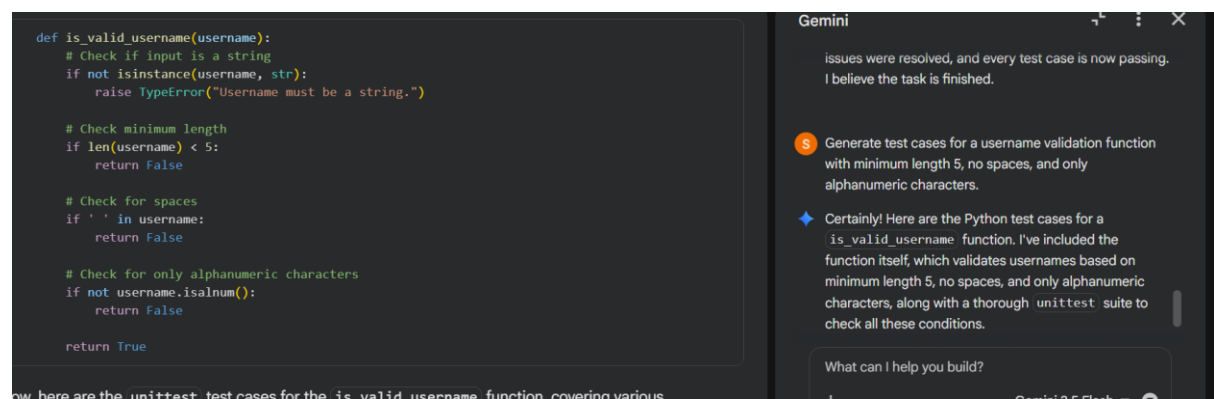
- Ensures marks are valid.

- Average calculated dynamically.
- Result determined based on average ≥ 40 .

Task 5 – Test-Driven Development for Username Validator

Prompt used:

Generate test cases for a username validation function with minimum length 5, no spaces, and only alphanumeric characters.



```

    with self.assertRaises(TypeError, msg="List should raise TypeError"):
        is_valid_username(["username"])
    with self.assertRaises(TypeError, msg="Boolean True should raise TypeError"):
        is_valid_username(True)

Run the tests in Colab
__name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False, verbosity=2)

test_negative_even_numbers (__main__.TestIsEven.test_negative_even_numbers) ... ok
test_negative_odd_numbers (__main__.TestIsEven.test_negative_odd_numbers) ... ok
test_non_integer_inputs (__main__.TestIsEven.test_non_integer_inputs) ... ok
test_positive_even_numbers (__main__.TestIsEven.test_positive_even_numbers) ... ok
test_positive_odd_numbers (__main__.TestIsEven.test_positive_odd_numbers) ... ok
test_very_large_numbers (__main__.TestIsEven.test_very_large_numbers) ... ok
test_zero (__main__.TestIsEven.test_zero) ... ok
test_non_string_inputs (__main__.TestIsValidUsername.test_non_string_inputs) ... ok
test_username_too_short (__main__.TestIsValidUsername.test_username_too_short) ... ok
test_username_with_spaces (__main__.TestIsValidUsername.test_username_with_spaces) ... ok
test_username_with_special_characters (__main__.TestIsValidUsername.test_username_with_special_characters) ... ok
test_valid_usernames (__main__.TestIsValidUsername.test_valid_usernames) ... ok
test_already_lowercase (__main__.TestStringConversion.test_already_lowercase) ... ok
test_already_uppercase (__main__.TestStringConversion.test_already_uppercase) ... ok
test_empty_string (__main__.TestStringConversion.test_empty_string) ... ok
test_mixed_case_to_lowercase (__main__.TestStringConversion.test_mixed_case_to_lowercase) ... ok
test_mixed_case_to_uppercase (__main__.TestStringConversion.test_mixed_case_to_uppercase) ... ok
test_non_string_inputs_lowercase (__main__.TestStringConversion.test_non_string_inputs_lowercase) ... ok
test_non_string_inputs_uppercase (__main__.TestStringConversion.test_non_string_inputs_uppercase) ... ok

```

```

test_get_result_borderline_pass (__main__.TestStudentResult.test_get_result_borderline_pass) ... ok
test_get_result_fail (__main__.TestStudentResult.test_get_result_fail) ... ok
test_get_result_no_marks (__main__.TestStudentResult.test_get_result_no_marks) ... ok
test_get_result_pass (__main__.TestStudentResult.test_get_result_pass) ... ok
test_empty_list (__main__.TestSumListElements.test_empty_list) ... ok
test_floats (__main__.TestSumListElements.test_floats) ... ok
test_large_numbers (__main__.TestSumListElements.test_large_numbers) ... ok
test_mixed_numbers (__main__.TestSumListElements.test_mixed_numbers) ... ok
test_mixed_numeric_and_non_numeric (__main__.TestSumListElements.test_mixed_numeric_and_non_numeric) ... ok
test_negative_numbers (__main__.TestSumListElements.test_negative_numbers) ... ok
test_only_non_numeric (__main__.TestSumListElements.test_only_non_numeric) ... ok
test_positive_numbers (__main__.TestSumListElements.test_positive_numbers) ... ok
test_zero (__main__.TestSumListElements.test_zero) ... ok

=====
FAIL: test_mixed_numeric_and_non_numeric (__main__.TestSumListElements.test_mixed_numeric_and_non_numeric)
-----
Traceback (most recent call last):
  File "/tmp/ipython-input-3362737179.py", line 33, in test_mixed_numeric_and_non_numeric
    self.assertEqual(sum_list_elements([1, 'a', 2, None, 3, True, 4.5]), 10.5, "Non-numeric values should be ignored")
AssertionError: 11.5 != 10.5 : Non-numeric values should be ignored

-----
Ran 39 tests in 0.077s

FAILED (failures=1)

```

Code Explanation

- Ensures string input.

- Checks minimum length.
- Prevents spaces.
- Allows only letters and numbers.